

# Binary Search Tree Insert Visualisation

Inserting: 1 5 3 9 4 16 12 2

1



**Insert 1 — empty tree:** `self == NULL` is true (root is NULL). `malloc` a new node, set `data_item = 1`, `left = right = NULL`. Return it — node(1) becomes the root.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

2



**Insert 5 — compare at node 1:**  $5 > 1 \rightarrow$  the `else if (n > self->data_item)` branch fires. Execute `self->right = insert_bst_node(self->right, 5)`. node(1)  $\rightarrow$  right is currently NULL.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

3



Insert 5 — NULL reached:  $self == NULL \rightarrow$  create node(5), return it. The call unwinds:  $node(1) \rightarrow right = node(5)$ . Node 5 is the right child of 1.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

4



Insert 3 — compare at node 1:  $3 > 1 \rightarrow$  go right. Recurse into the right subtree rooted at node 5.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

5

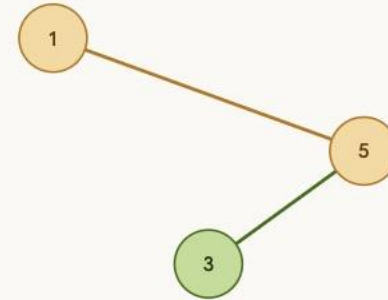


Insert 3 — compare at node 1:  $3 > 1 \rightarrow$  go right. Recurse into the right subtree rooted at node 5.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

6

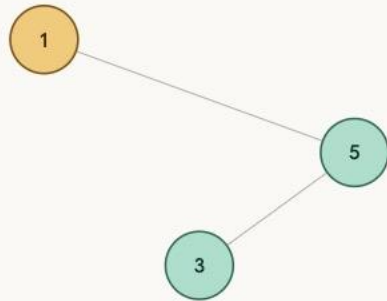


Insert 3 — NULL reached: Create node(3), return it. Unwind: node(5)→left = node(3). Node 3 is the left child of 5.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

7

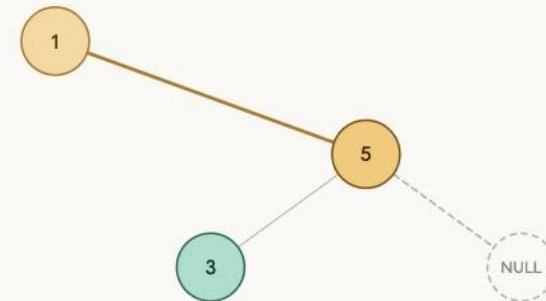


Insert 9 — compare at node 1:  $9 > 1 \rightarrow$  go right. Recurse into node 5.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

8

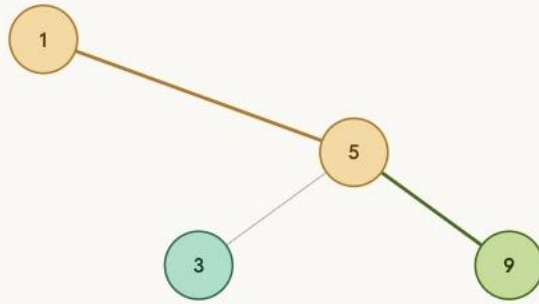


Insert 9 — compare at node 5:  $9 > 5 \rightarrow$  go right. Execute `self->right = insert_bst_node(self->right, 9)`.  
node(5)→right is NULL.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

9

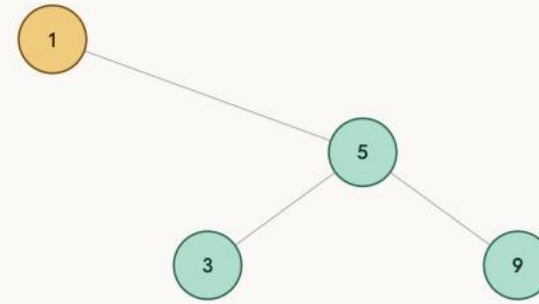


Insert 9 — NULL reached: Create node(9), return it. Unwind: node(5)→right = node(9). Node 9 is the right child of 5.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

10

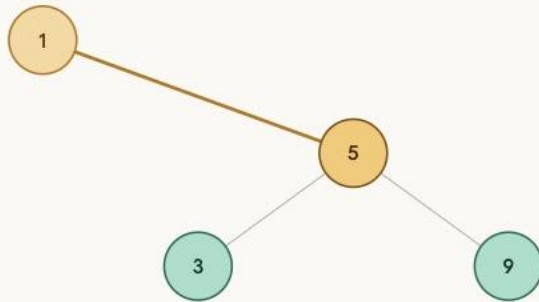


Insert 4 — compare at node 1: 4 > 1 → go right. Recurse into node 5.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

11

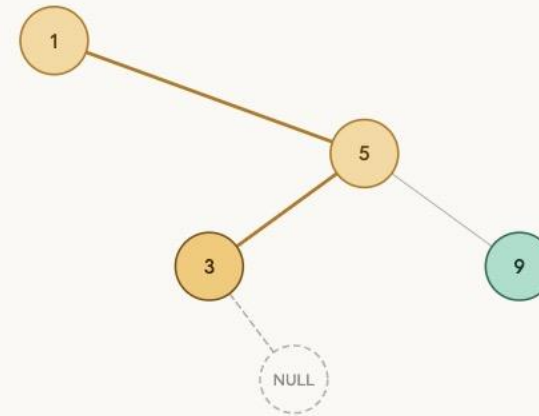


Insert 4 — compare at node 5:  $4 < 5 \rightarrow$  go left. Recurse into node 3.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

12

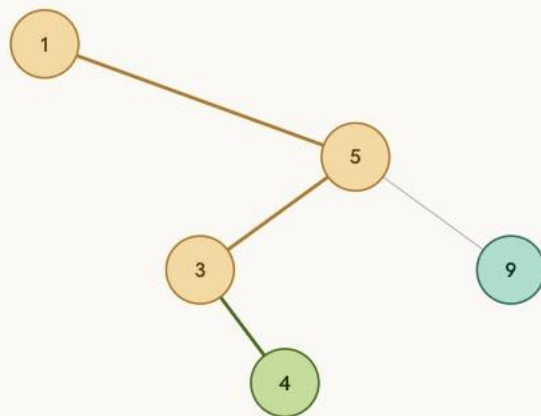


Insert 4 — compare at node 3:  $4 > 3 \rightarrow$  go right. Execute `self->right = insert_bst_node(self->right, 4)`. `node(3)->right` is NULL.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

13

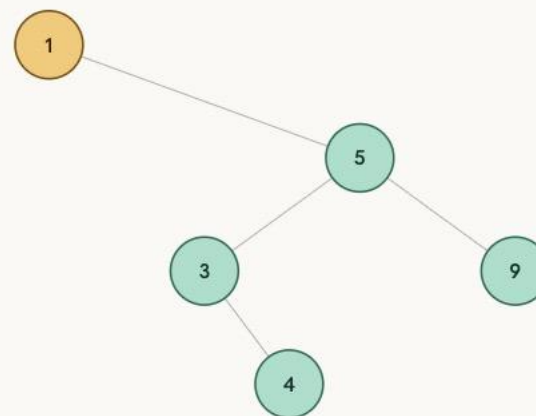


Insert 4 — NULL reached: Create node(4), return it. Unwind: node(3)→right = node(4). Node 4 is the right child of 3.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

14



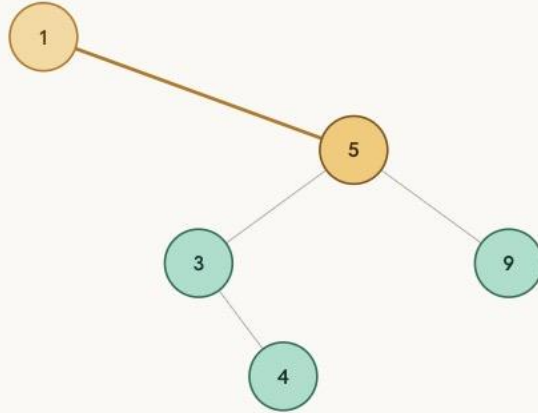
Insert 16 — compare at node 1: 16 > 1 → go right. Recurse into node 5.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```



Inserting: 1 5 3 9 4 16 12 2

15

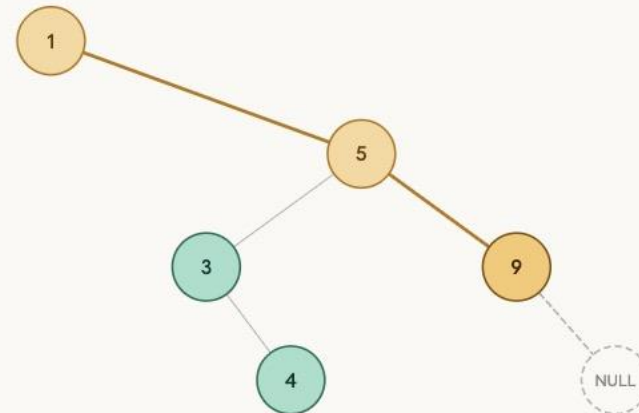


Insert 16 — compare at node 5:  $16 > 5 \rightarrow$  go right. Recurse into node 9.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

16

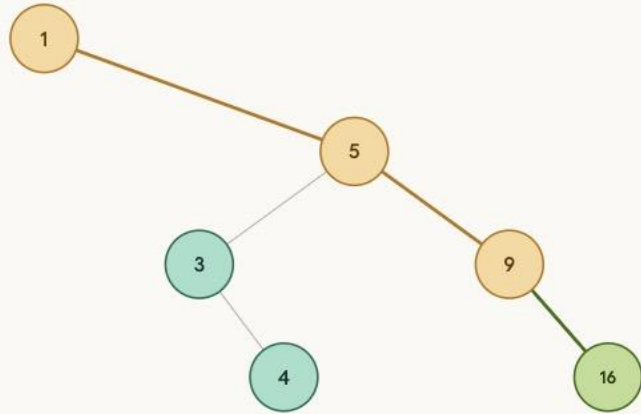


Insert 16 — compare at node 9:  $16 > 9 \rightarrow$  go right. Execute `self->right = insert_bst_node(self->right, 16)`. `node(9)->right` is NULL.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

17

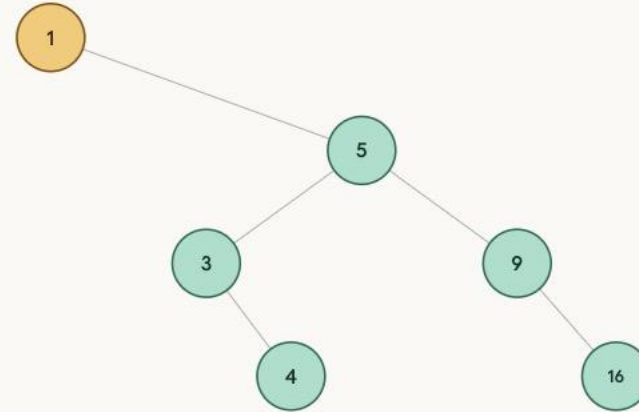


Insert 16 — NULL reached: Create node(16), return it. Unwind: node(9)→right = node(16). Node 16 is the right child of 9.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

18

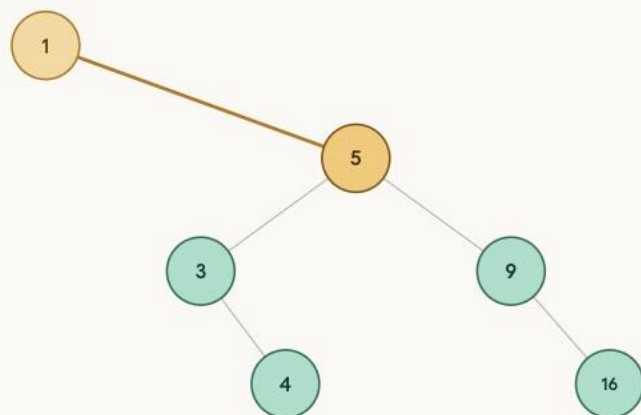


Insert 12 — compare at node 1: 12 > 1 → go right. Recurse into node 5.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

19

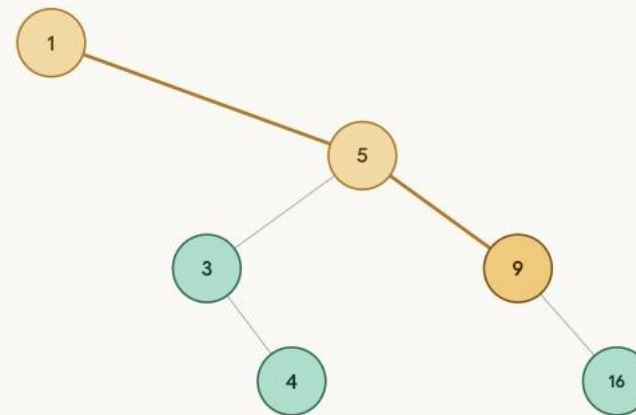


Insert 12 — compare at node 5:  $12 > 5 \rightarrow$  go right. Recurse into node 9.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

20

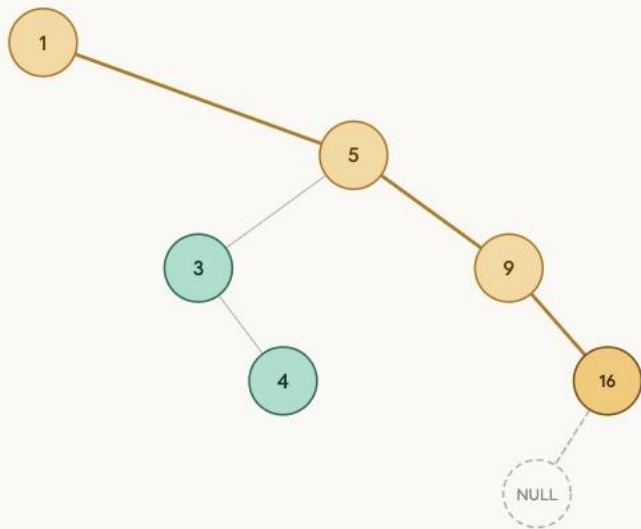


Insert 12 — compare at node 9:  $12 > 9 \rightarrow$  go right. Recurse into node 16.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

21

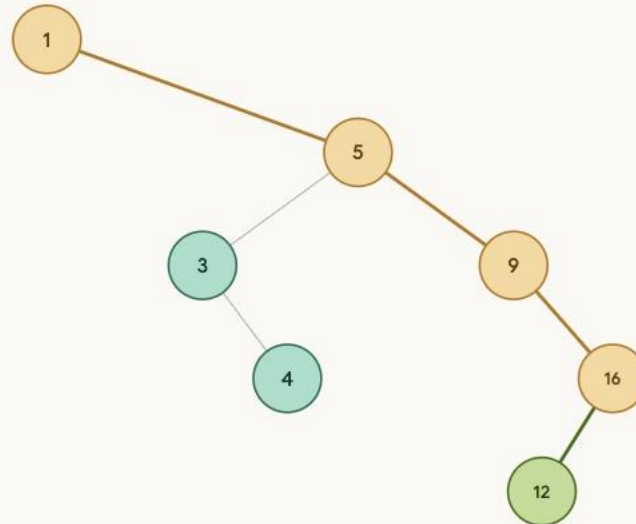


Insert 12 — compare at node 16:  $12 < 16 \rightarrow$  go left. Execute `self->left = insert_bst_node(self->left, 12)`. `node(16)->left` is NULL.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

22

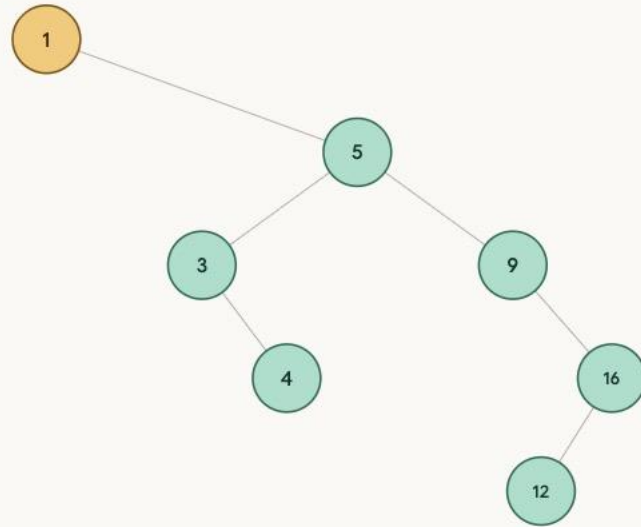


Insert 12 — NULL reached: Create node(12), return it. Unwind: `node(16)->left = node(12)`. Node 12 is the left child of 16.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

23

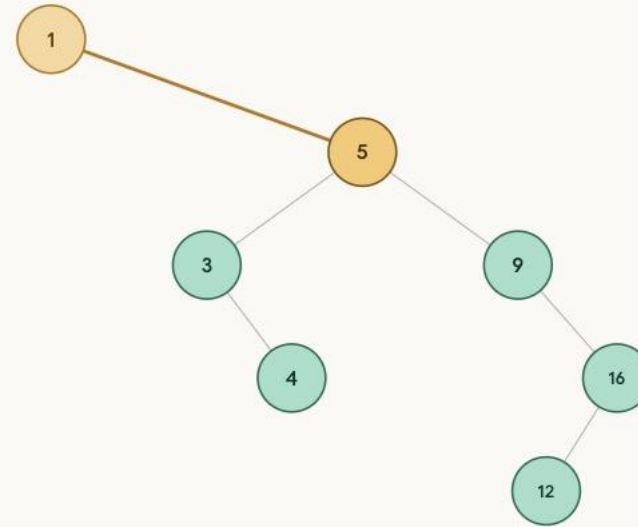


Insert 2 — compare at node 1:  $2 > 1 \rightarrow$  go right. Recurse into node 5.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

24

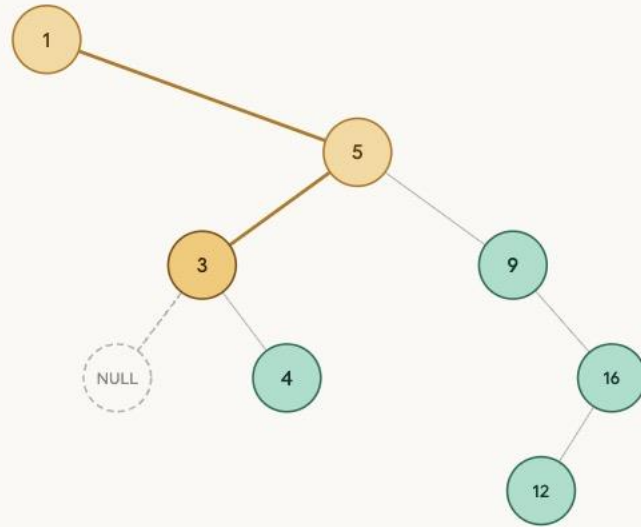


Insert 2 — compare at node 5:  $2 < 5 \rightarrow$  go left. Recurse into node 3.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

25

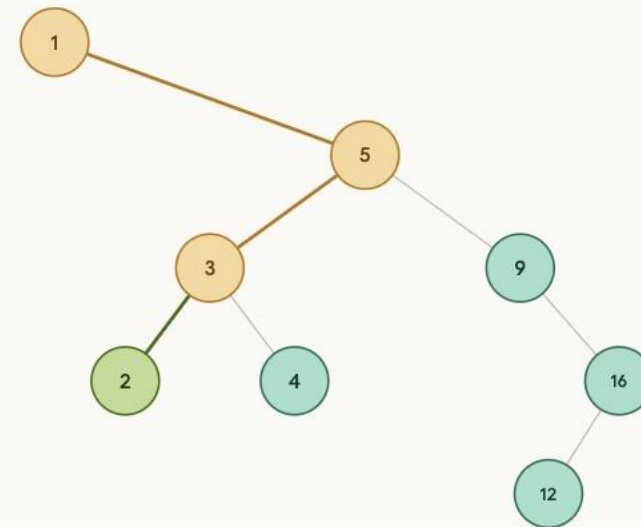


Insert 2 — compare at node 3:  $2 < 3 \rightarrow$  go left. Execute `self->left = insert_bst_node(self->left, 2)`. `node(3)->left` is NULL.

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```

Inserting: 1 5 3 9 4 16 12 2

26



Insert 2 — NULL reached: Create node(2), return it. Unwind: `node(3)->left = node(2)`. Node 2 is the left child of 3. All 8 values inserted!

```
BSTNodePtr insert_bst_node(BSTNodePtr self, int n) {
    if (self == NULL) {
        self = malloc(sizeof *self);
        self->data_item = n;
        self->left = self->right = NULL;
    } else if (n < self->data_item) {
        self->left = insert_bst_node(self->left, n);
    } else if (n > self->data_item) {
        self->right = insert_bst_node(self->right, n);
    }
    return self;
}
```